

CLASE PRACTICA

Informe de desarrollo y evidencias | Diseño de sistemas en Internet

NestJS · Prisma · SQLite · CRUD · Swagger · JWT

5 de octubre de 2026 | Nombre del estudiante: por completar

Práctica de Diseño de sistemas en Internet, desarrollada el 5 de octubre de 2026. Se integraron las tres guías proporcionadas: creación del ecosistema y persistencia, CRUD de usuarios y autenticación JWT. El resultado es una API local funcional con NestJS 11, Prisma 6.19.3, SQLite y Swagger. Se conservaron capturas auténticas de Visual Studio Code y del navegador, sin la aplicación ChatGPT visible en ellas. Los comandos se ejecutaron mediante herramientas de terminal y sus salidas se conservaron como archivos de texto; las capturas de esos registros dentro del editor no son capturas de una terminal integrada. No se consiguió una imagen conjunta de editor y terminal en cada paso. Las imágenes evidencian estados reales del desarrollo y la validación, no cada pulsación ni una autoría manual del estudiante. Los comandos de generación de archivos se ofrecen como referencia reproducible: el esquema y los módulos se escribieron directamente. La captura de versiones corresponde a una verificación final del entorno.

Resultado verificado: 19 pruebas correctas, dos migraciones aplicadas y tres commits locales.

Proyecto: clase practica | API local: <http://127.0.0.1:3001/api>

Ruta de trabajo

Guía 1: entorno, estructura del proyecto, modelo User/Tenant, migraciones y seeder.

Guía 2: PrismaService, inyección de dependencias, DTO, operaciones CRUD y Swagger.

Guía 3: login, hash de contraseñas, firma JWT, estrategia Passport, guard y Bearer en Swagger.

Verificación: arranque, 19 pruebas, migraciones al día y conservación de evidencias.

La carpeta src contiene la aplicación; prisma contiene persistencia y datos iniciales; test contiene la verificación HTTP. dist es la salida compilada y node_modules contiene las dependencias.

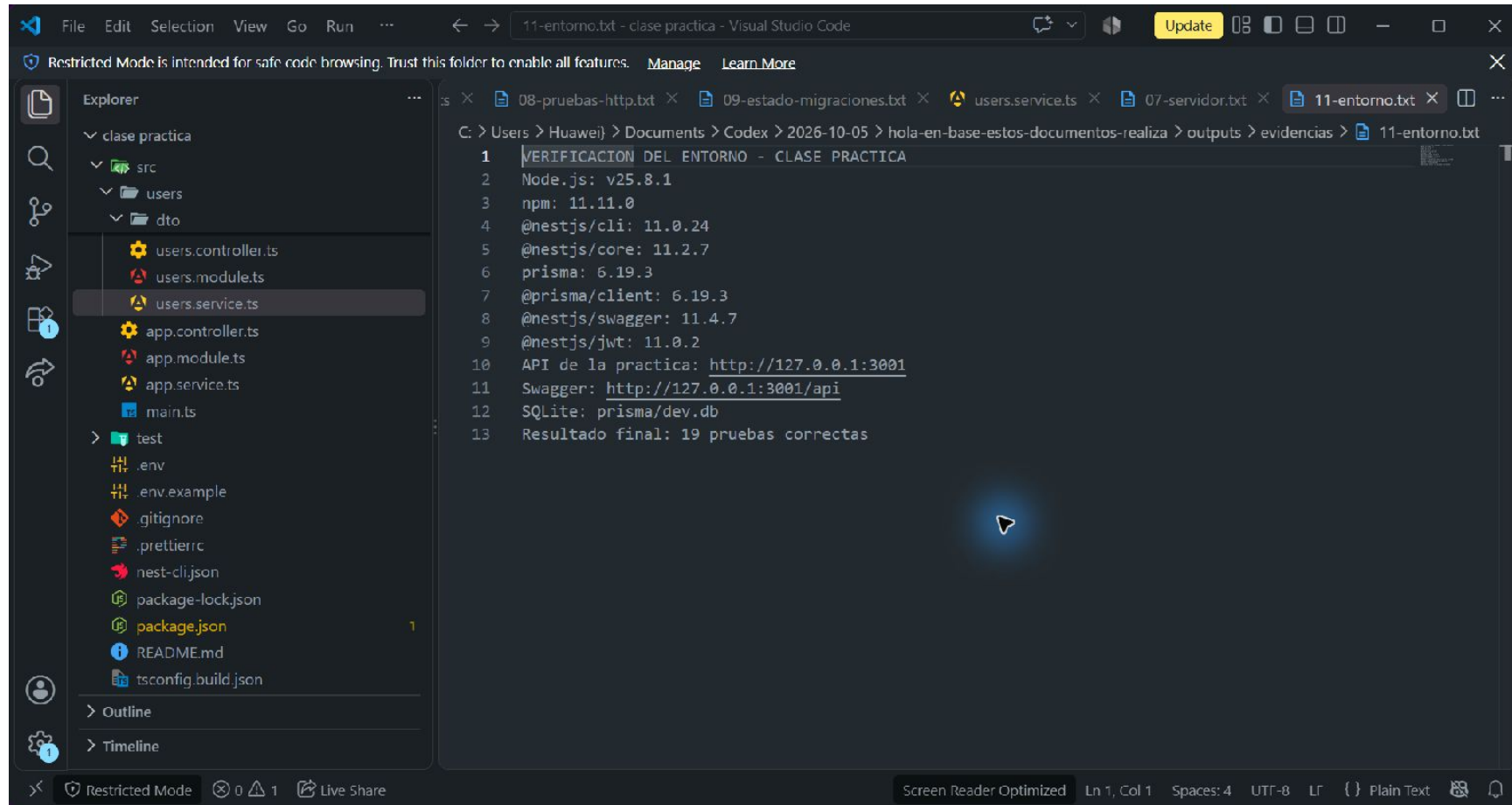
Las páginas siguientes describen el paso a paso y muestran capturas reales. Algunas evidencias corresponden al código o al registro de salida visible en el editor; no implican que cada comando se haya escrito en su terminal integrada.

01 Preparar el entorno y abrir el proyecto

Guía 1, páginas 1-3

Se comprobó Node.js v25.8.1 y npm 11.11.0. Se abrió una ventana de Visual Studio Code para la carpeta clase practica. El nombre solicitado se conservó en la carpeta; package.json utiliza clase-practica, sin espacios.

```
node --version
npm --version
```



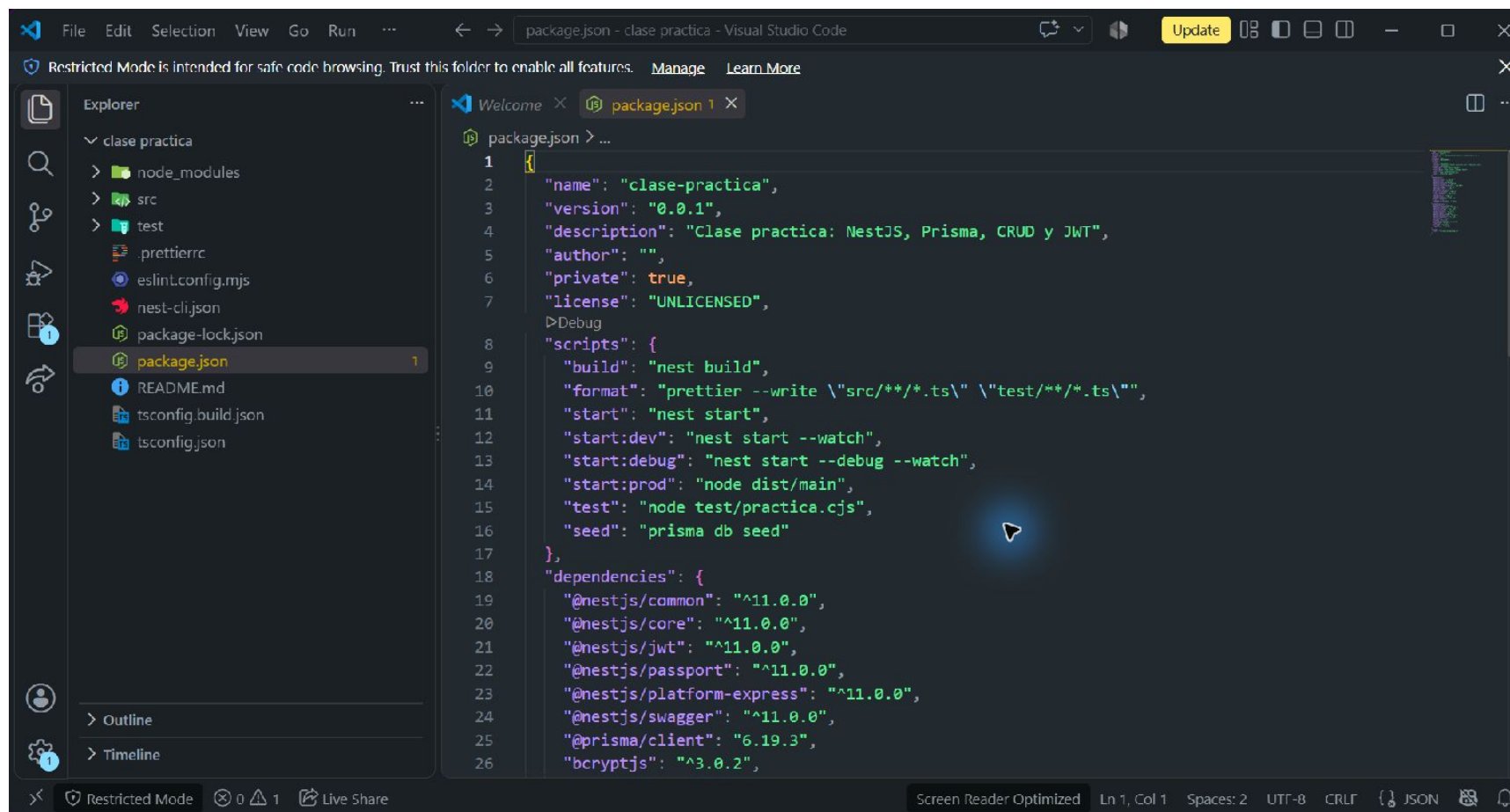
Captura real de ventana o navegador. Archivo: 20-entorno.png

02 Crear la plantilla e instalar dependencias

Guía 1, páginas 3-6

Se generó una plantilla real con Nest CLI 11.0.24. Se utilizó una instalación existente del CLI para evitar una descarga lenta; las dependencias se instalaron con npm y su caché local. No se reutilizó el código de otro proyecto. Se fijó Prisma 6.19.3 para conservar la configuración de las guías.

```
nest new clase-practica --directory "clase practica" --package-manager npm --skip-install --skip-git --strict
npm install
```



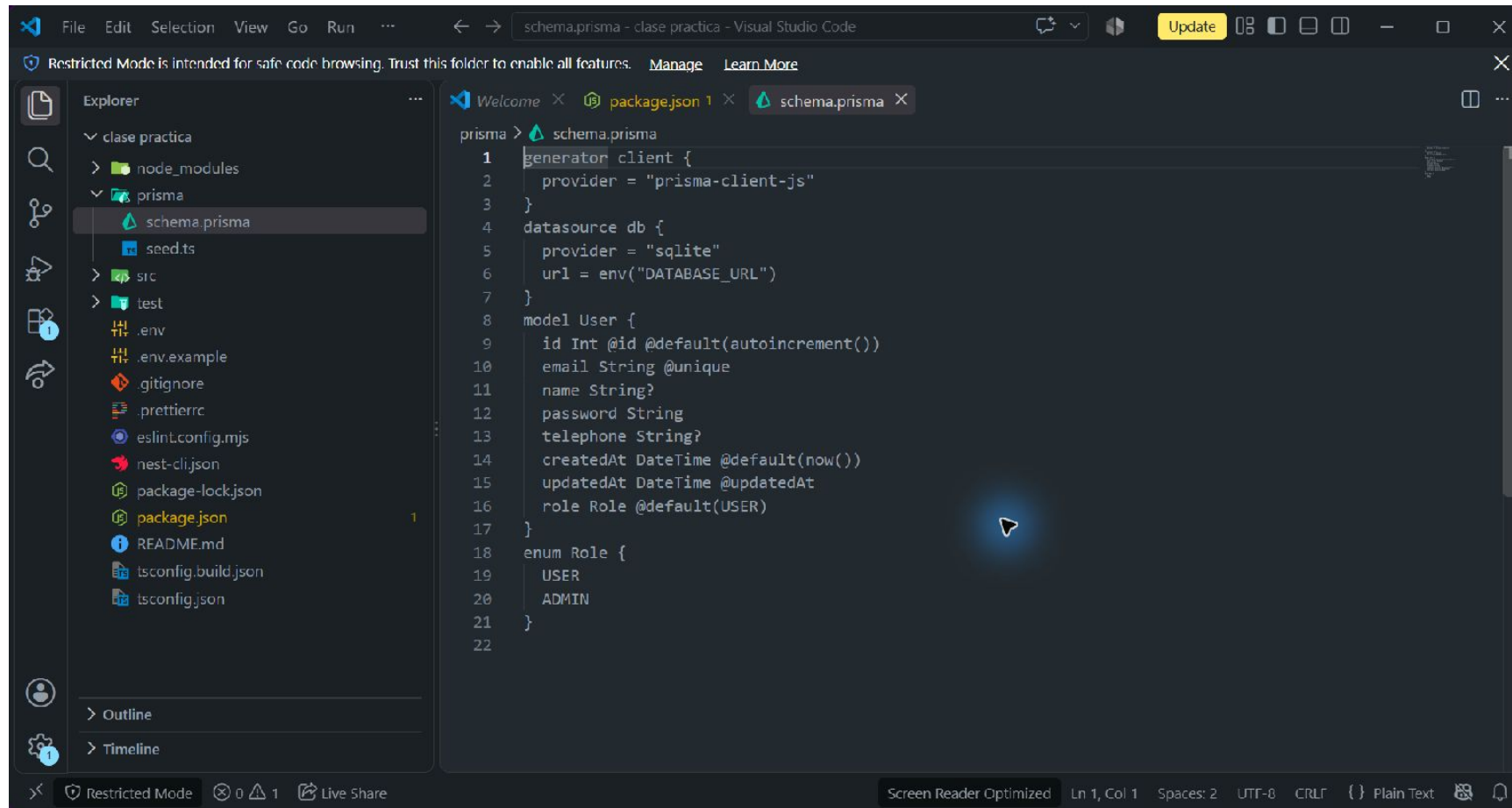
Captura real de ventana o navegador. Archivo: 02-proyecto.png

03 Configurar Prisma y el modelo User

Guía 1, páginas 9-12

En prisma/schema.prisma se definieron generator, datasource SQLite y User. El usuario contiene email único, nombre opcional, contraseña, teléfono, fechas y rol USER o ADMIN. La variable DATABASE_URL apunta a file:./dev.db. El secreto JWT se mantiene en .env y no aparece en las capturas.

```
npm install --save-dev prisma@6.19.3
npm install @prisma/client@6.19.3
npx prisma init --datasource-provider sqlite
```



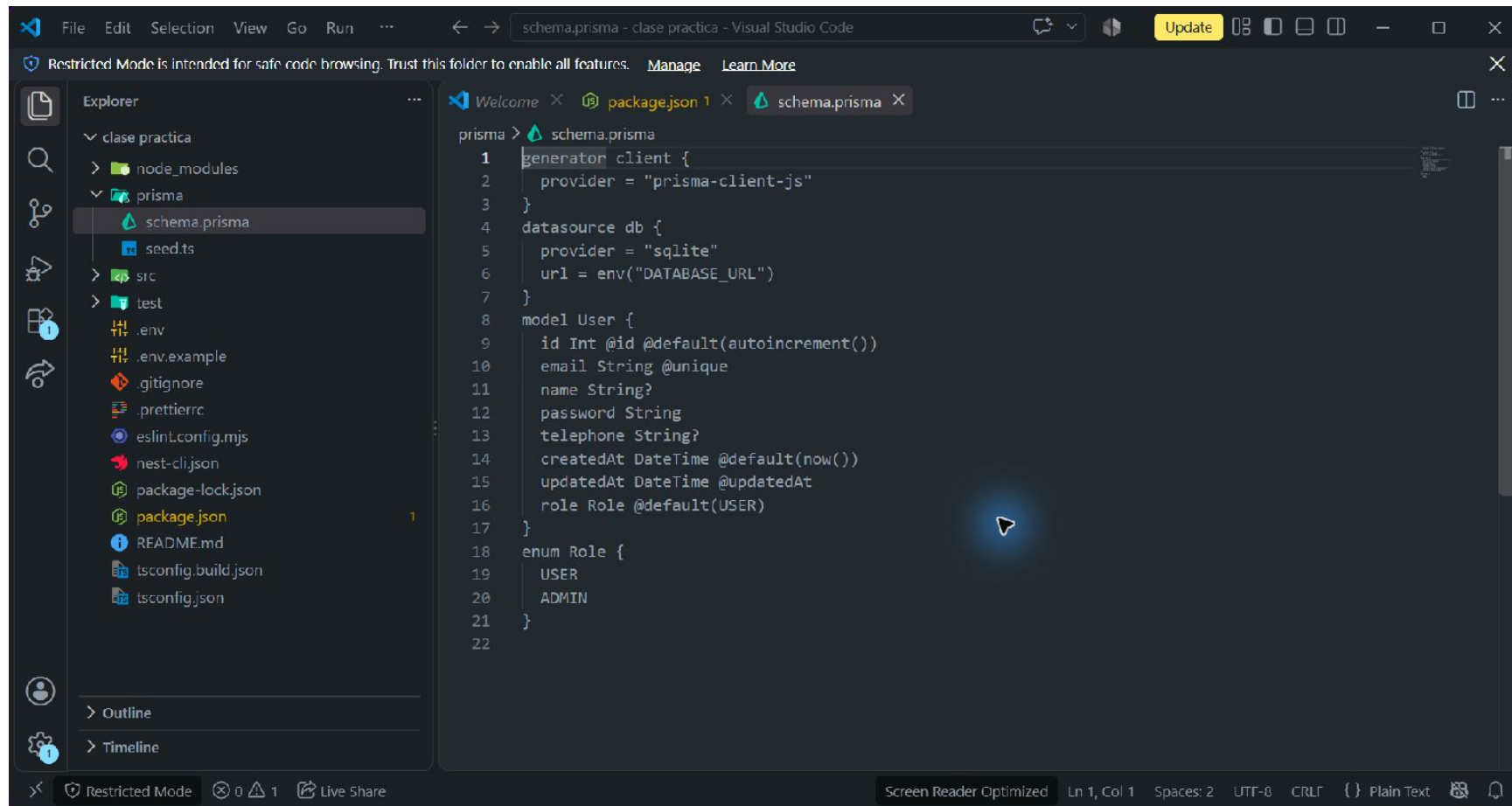
Captura real de ventana o navegador. Archivo: 03-schema-inicial.png

04 Aplicar la migración inicial

Guía 1, página 12

Se ejecutó `migrate dev --name init` y se creó la migración `20261005185707_init`. El primer intento devolvió `Schema engine error`. La solución fue crear un archivo SQLite vacío y repetir la operación. El registro conservado confirma que la migración se aplicó y se generó Prisma Client.

```
npx prisma migrate dev --name init
```



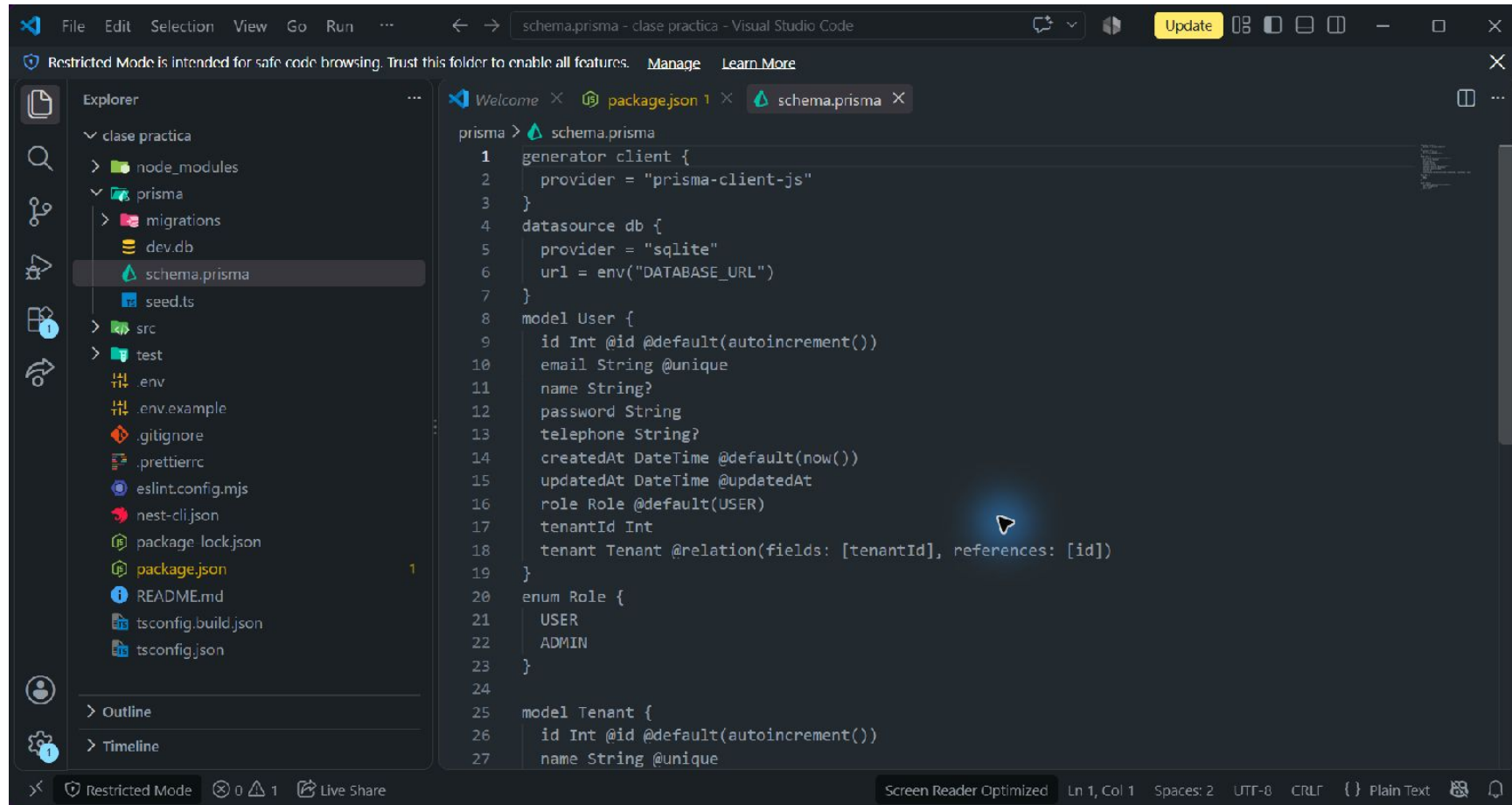
Captura real de ventana o navegador. Archivo: 03-schema-inicial.png

05 Relacionar User con Tenant

Guía 1, páginas 13-15

Se agregó Tenant con id, name único y users User[]. User recibe tenantId y la relación mediante fields y references. Un tenant puede tener muchos usuarios; cada usuario pertenece a un tenant. Se aplicó 20261005185725_add_tenant_model.

```
npx prisma migrate dev --name add_tenant_model
```



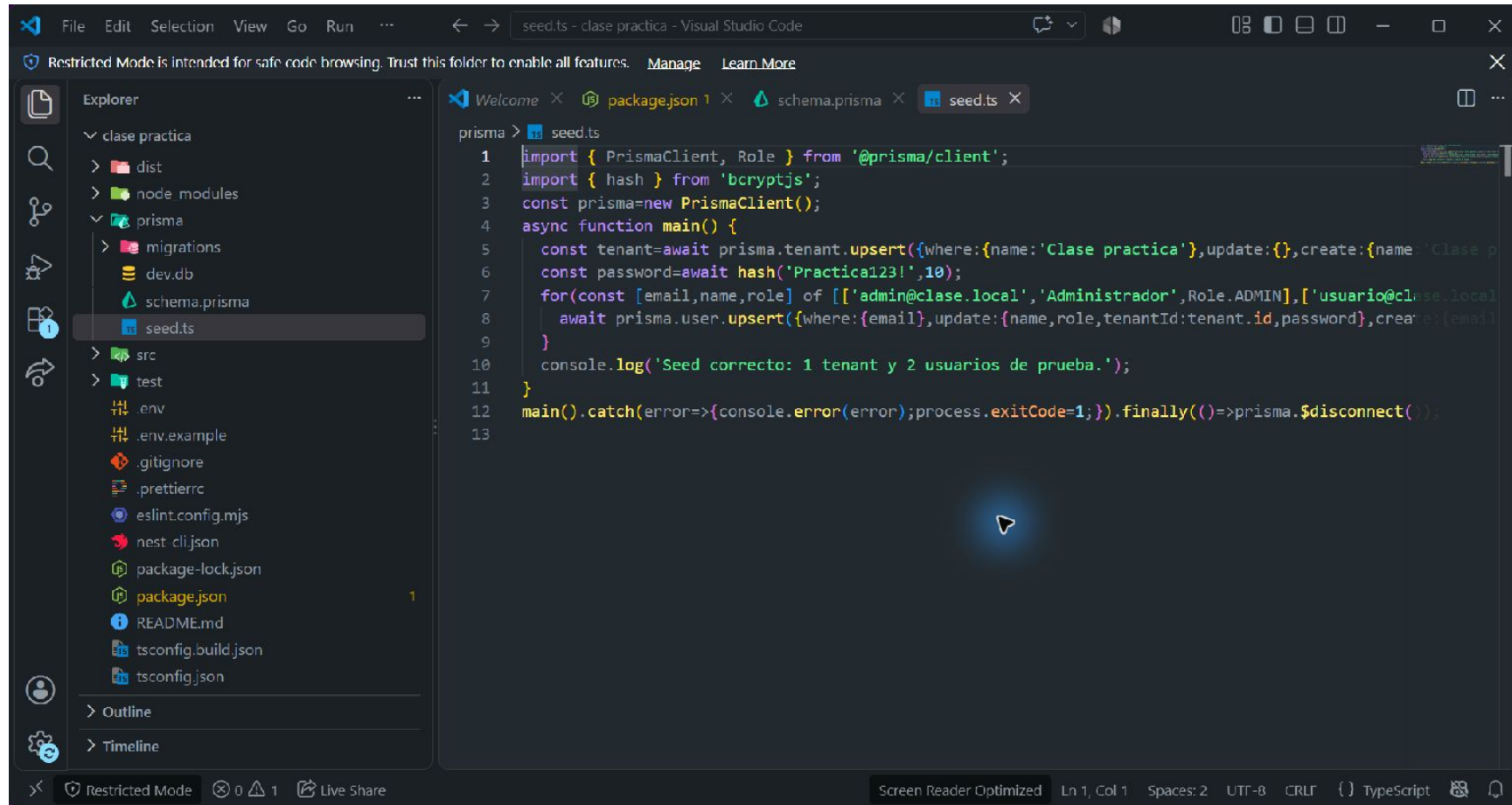
Captura real de ventana o navegador. Archivo: 04-relacion-tenant.png

06 Insertar datos iniciales con el seeder

Guía 1, páginas 16-17

prisma/seed.ts utiliza upsert para un tenant y dos usuarios de demostración. bcryptjs genera un hash con costo 10. Los usuarios son admin@clase.local y usuario@clase.local, con contraseña de práctica Practica123!. El registro confirmó: 1 tenant y 2 usuarios.

```
npm install bcryptjs
npx prisma db seed
```



```
prisma > seed.ts
1 import { PrismaClient, Role } from '@prisma/client';
2 import { hash } from 'bcryptjs';
3 const prisma = new PrismaClient();
4 async function main() {
5   const tenant = await prisma.tenant.upsert({where: {name: 'Clase practica'}, update: {}, create: {name: 'Clase p
6   const password = await hash('Practica123!', 10);
7   for(const [email, name, role] of [['admin@clase.local', 'Administrador', Role.ADMIN], ['usuario@clase.local
8     await prisma.user.upsert({where: {email}, update: {name, role, tenantId: tenant.id, password}, create: {email
9   }
10  console.log('Seed correcto: 1 tenant y 2 usuarios de prueba.');
```

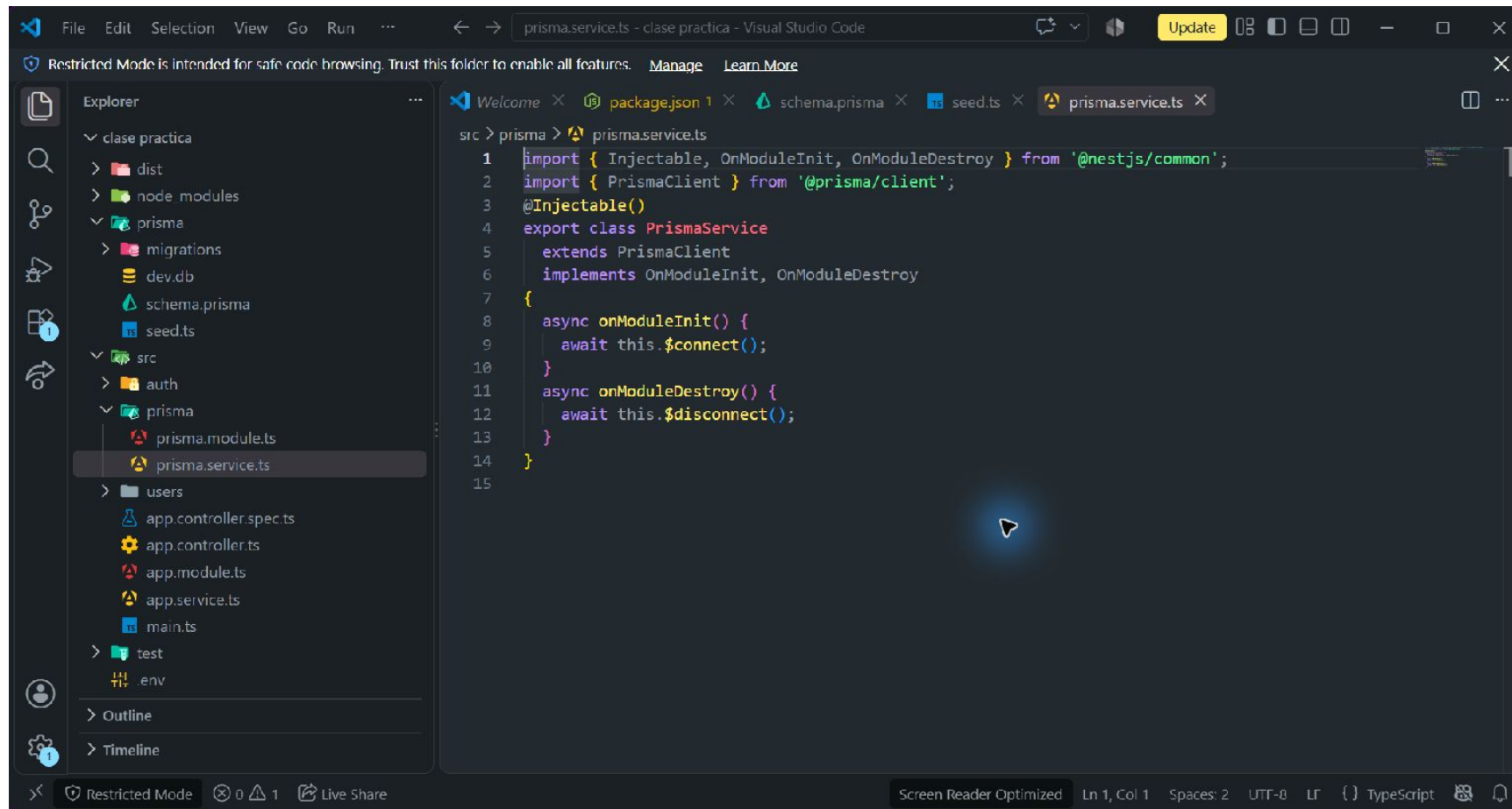
Captura real de ventana o navegador. Archivo: 05-seeder.png

07 Crear PrismaModule y PrismaService

Guía 2, página 1

PrismaService extiende PrismaClient y conecta o desconecta durante el ciclo de vida de Nest. PrismaModule exporta el servicio para compartir la misma instancia por inyección de dependencias. UsersModule y AuthModule importan PrismaModule. Los archivos se implementaron directamente, con la estructura equivalente a los generadores.

```
nest generate module prisma
nest generate service prisma
```



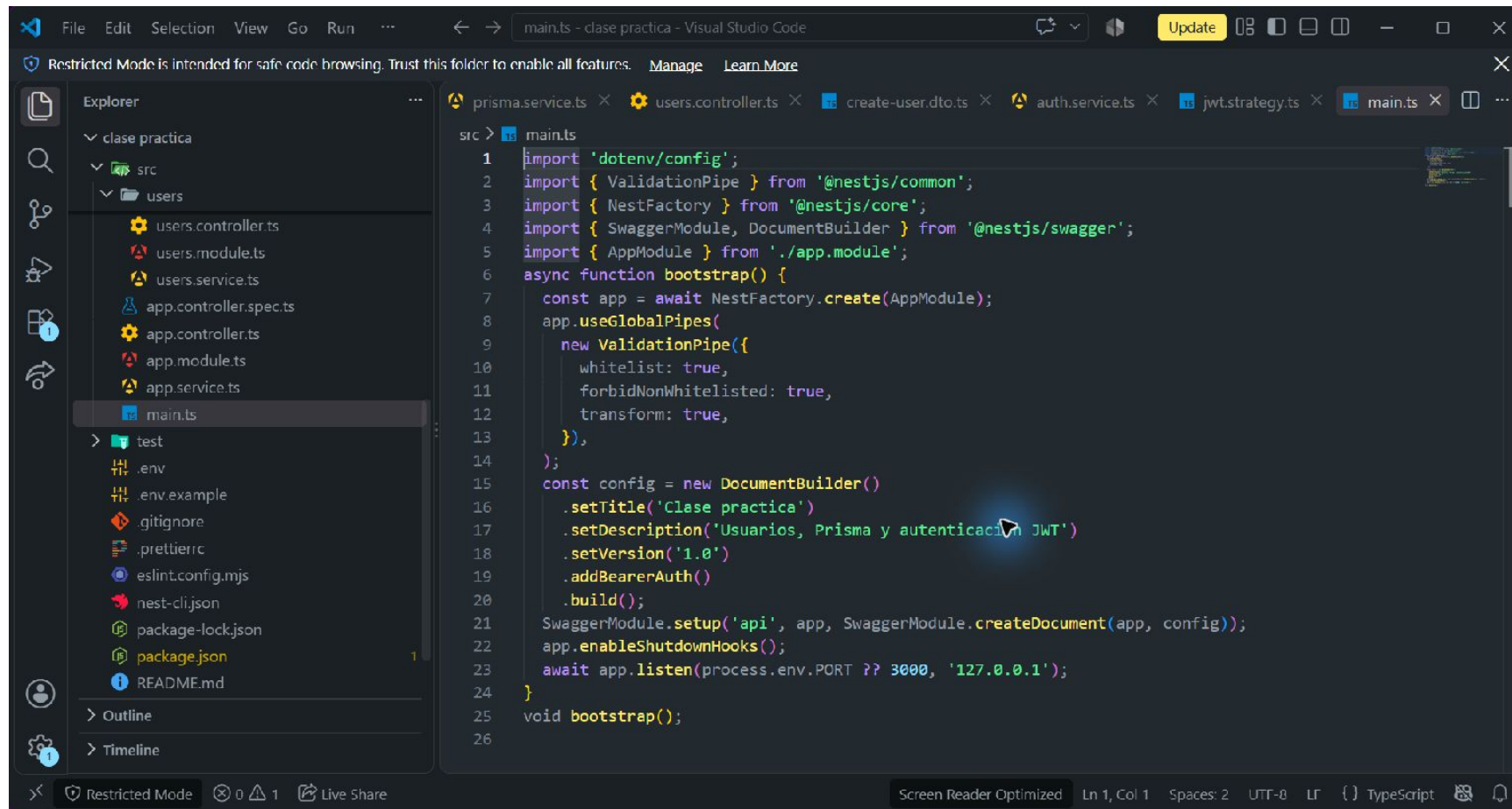
Captura real de ventana o navegador. Archivo: 06-prisma-service.png

08 Configurar Swagger y validación global

Guía 2, página 2; Guía 3, páginas 9-11

main.ts registra Swagger en /api, define el título Clase practica y configura Bearer JWT. ValidationPipe verifica DTO, elimina la aceptación de propiedades no declaradas y transforma parámetros. La aplicación escucha solo en 127.0.0.1.

```
npm install @nestjs/swagger swagger-ui-express class-validator class-transformer
http://127.0.0.1:3001/api
```



Captura real de ventana o navegador. Archivo: 15-main-swagger.png

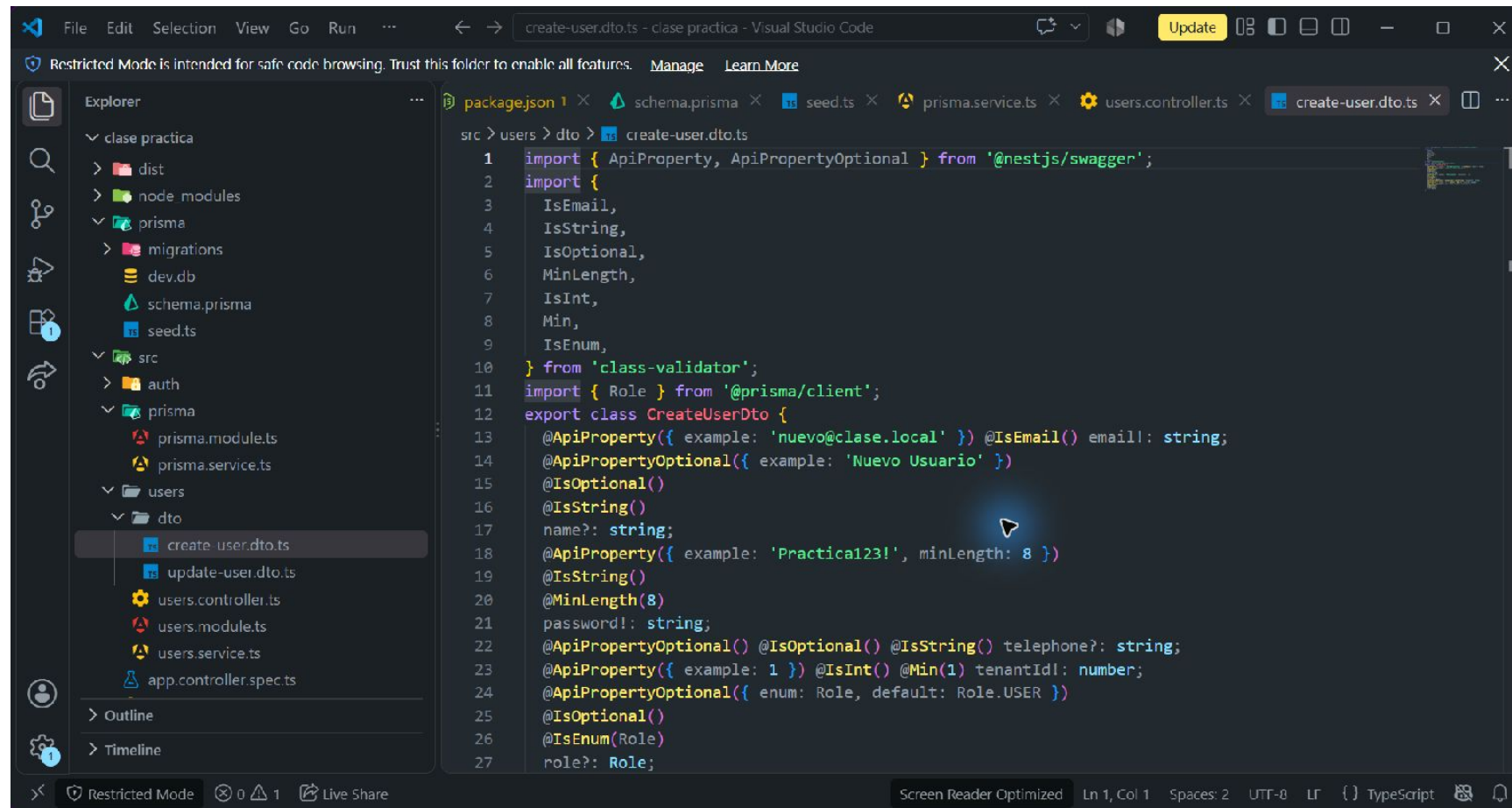
09 Definir los DTO de usuarios

Guía 2, páginas 5-6

CreateUserDto valida email, contraseña de al menos ocho caracteres, tenantId entero positivo y rol. UpdateUserDto hereda el contrato mediante PartialType, haciendo opcionales los campos para PATCH. Swagger documenta sus propiedades y ejemplos.

Crear src/users/dto/create-user.dto.ts

Crear src/users/dto/update-user.dto.ts



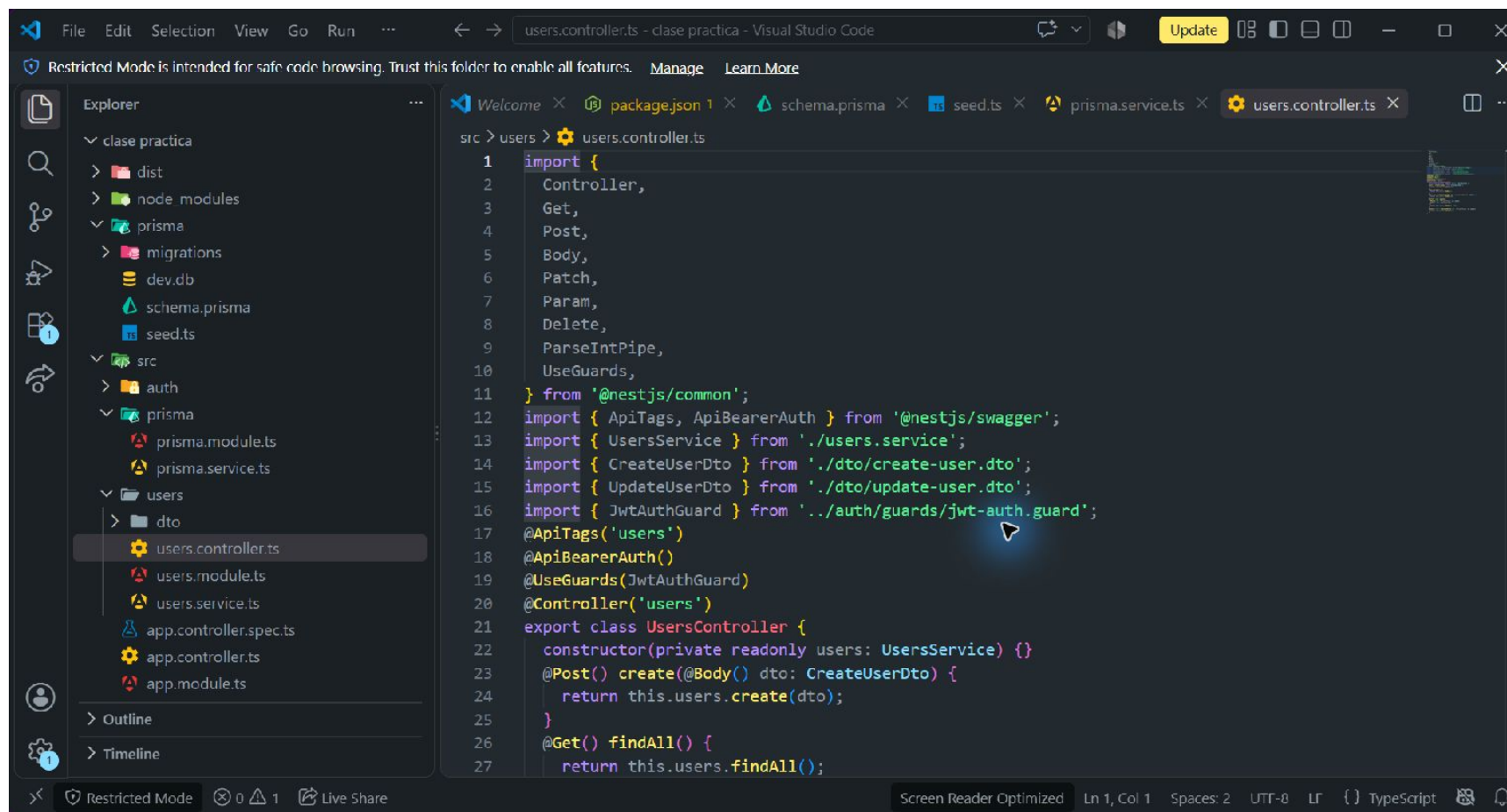
Captura real de ventana o navegador. Archivo: 08-dto.png

10 Implementar las cinco rutas CRUD

Guía 2, páginas 3-6

UserController expone POST /users, GET /users, GET /users/:id, PATCH /users/:id y DELETE /users/:id. ParseIntPipe valida el id. El controlador delega al servicio; JwtAuthGuard protege todo el recurso /users en la versión final.

POST /users | GET /users | GET /users/:id
PATCH /users/:id | DELETE /users/:id



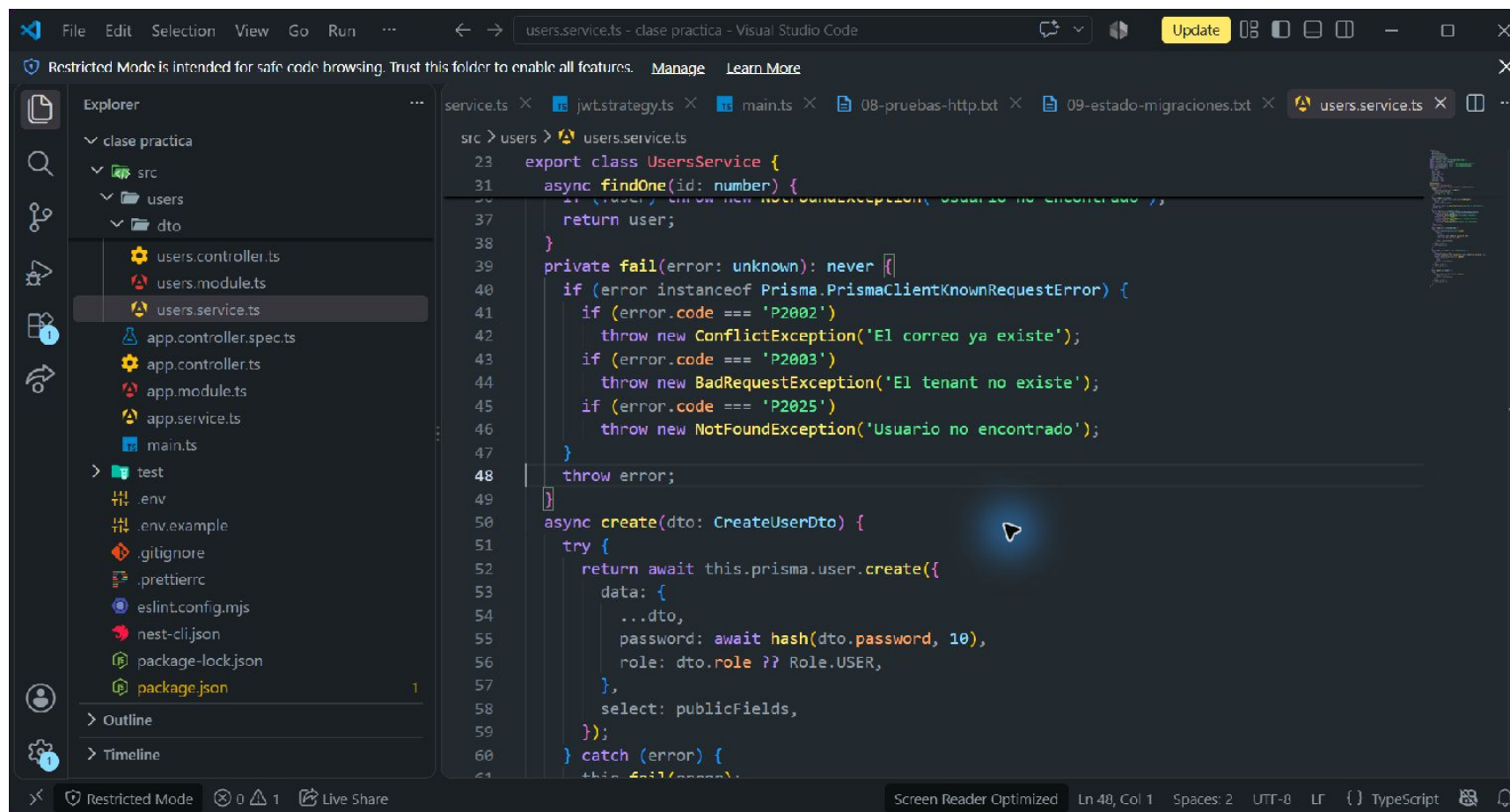
Captura real de ventana o navegador. Archivo: 07-crud-controller.png

11 Persistir las operaciones con Prisma

Guía 2, páginas 4-6

UserService utiliza findMany, findUnique, create, update y delete. Las contraseñas nuevas y modificadas se transforman en hashes bcrypt, sin cifrado reversible. Las respuestas seleccionan campos públicos y excluyen password. Correo duplicado devuelve 409, tenant inexistente 400 y usuario ausente 404.

findMany / findUnique / create / update / delete



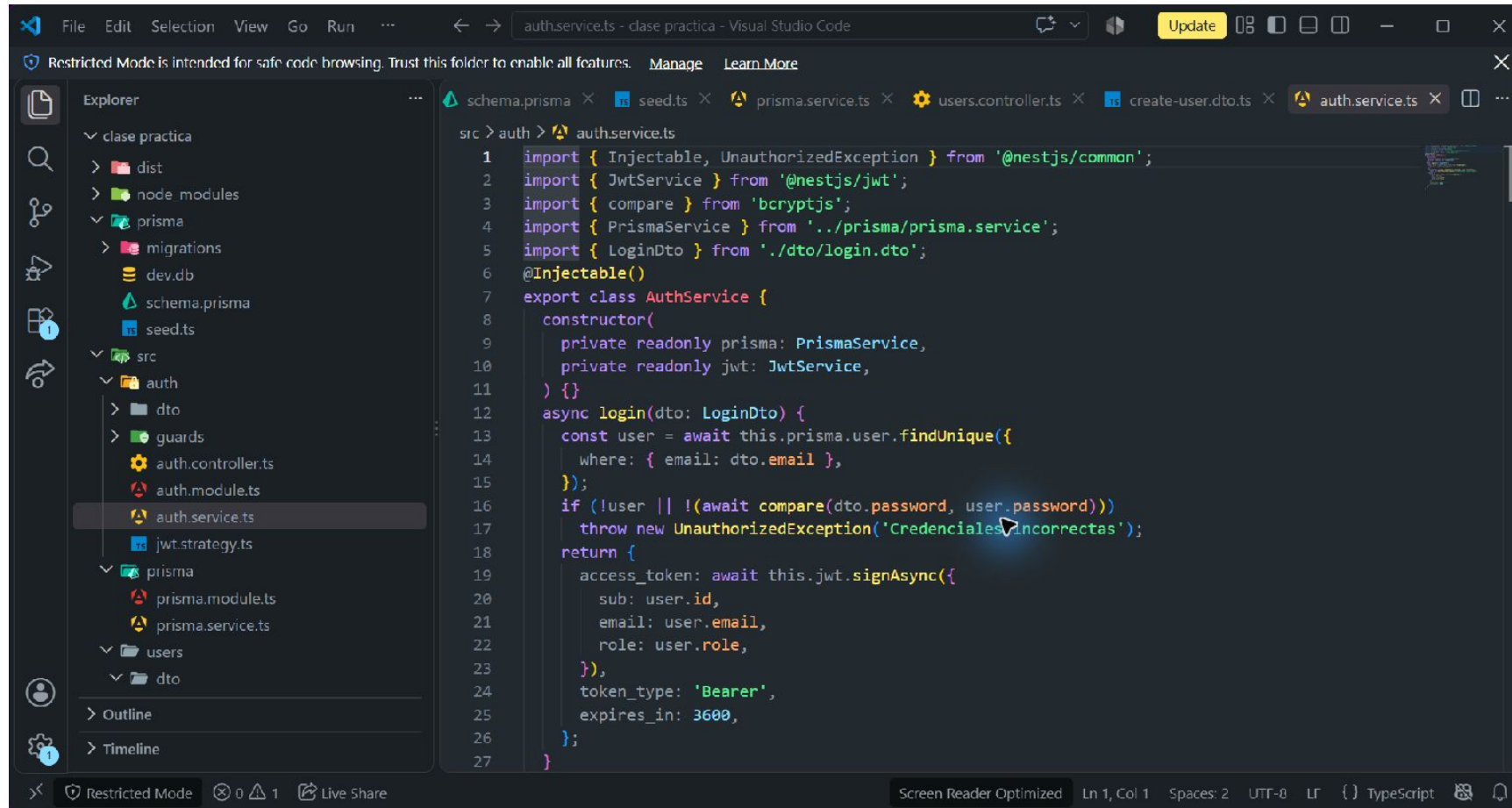
Captura real de ventana o navegador. Archivo: 18-crud-service.png

12 Crear el login y firmar el JWT

Guía 3, páginas 1-7

Se agregaron AuthModule, AuthService, AuthController y LoginDto. POST /auth/login recibe email y password. El servicio busca el usuario, compara bcrypt y firma un token con sub, email y role; expira en 3600 segundos. Un error de credenciales devuelve 401.

```
npm install @nestjs/jwt @nestjs/passport passport passport-jwt
npm install --save-dev @types/passport-jwt
```



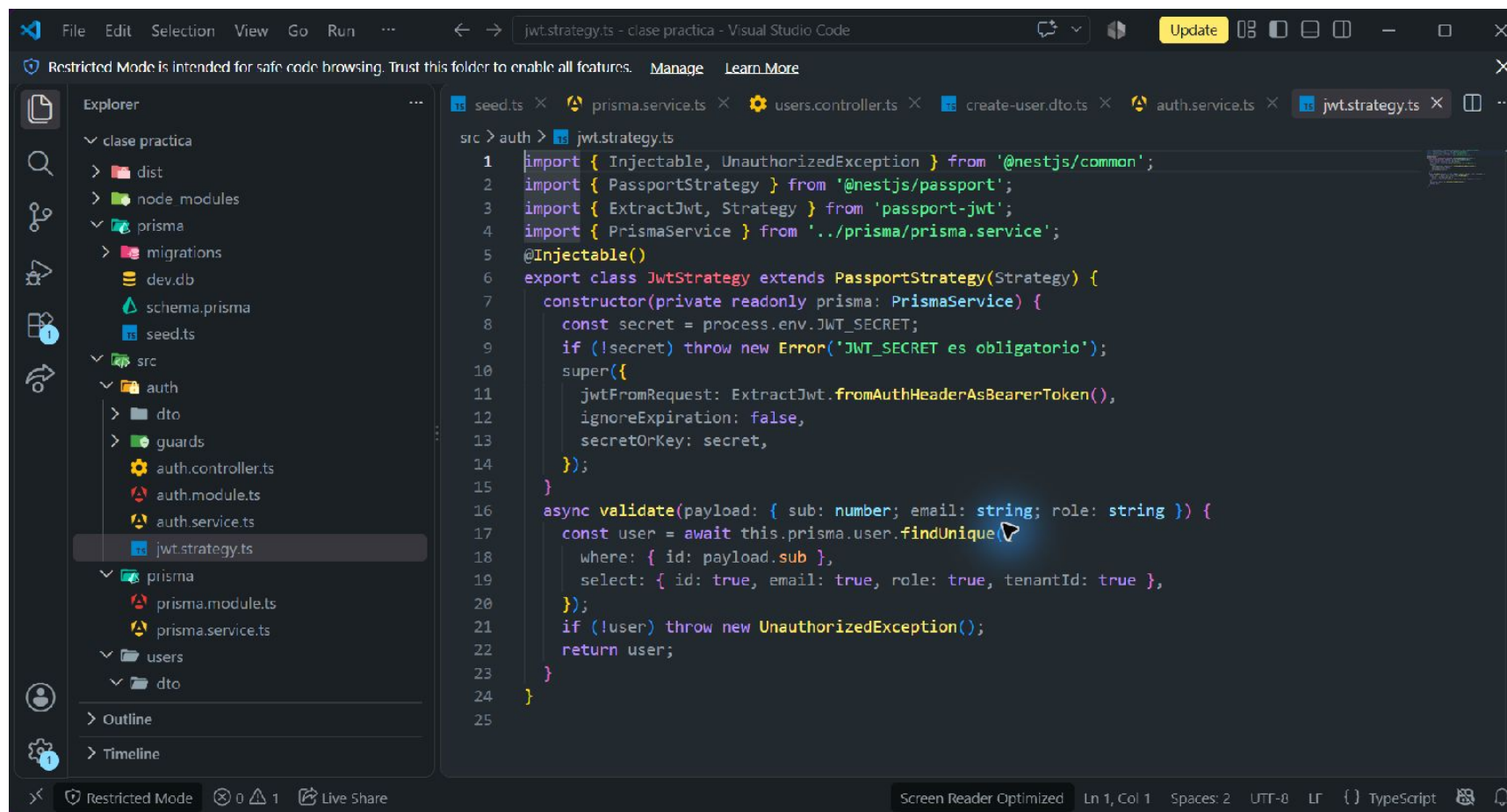
Captura real de ventana o navegador. Archivo: 09-auth-service.png

13 Verificar JWT y aplicar el guard

Guía 3, páginas 8-9

JwtStrategy extrae el token del encabezado Authorization: Bearer. Verifica la firma y la expiración; después comprueba que el usuario todavía existe. JwtAuthGuard extiende AuthGuard de Passport. JWT_SECRET se carga desde .env y es obligatorio.

```
Authorization: Bearer <access_token>  
@UseGuards(JwtAuthGuard)
```



```
1 import { Injectable, UnauthorizedException } from '@nestjs/common';  
2 import { PassportStrategy } from '@nestjs/passport';  
3 import { ExtractJwt, Strategy } from 'passport-jwt';  
4 import { PrismaService } from '../prisma/prisma.service';  
5 @Injectable()  
6 export class JwtStrategy extends PassportStrategy(Strategy) {  
7   constructor(private readonly prisma: PrismaService) {  
8     const secret = process.env.JWT_SECRET;  
9     if (!secret) throw new Error('JWT_SECRET es obligatorio');  
10    super({  
11      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),  
12      ignoreExpiration: false,  
13      secretOrKey: secret,  
14    });  
15  }  
16  async validate(payload: { sub: number; email: string; role: string }) {  
17    const user = await this.prisma.user.findUnique(  
18      where: { id: payload.sub },  
19      select: { id: true, email: true, role: true, tenantId: true },  
20    );  
21    if (!user) throw new UnauthorizedException();  
22    return user;  
23  }  
24 }  
25
```

Captura real de ventana o navegador. Archivo: 14-jwt-strategy.png

14 Compilar y levantar la aplicación

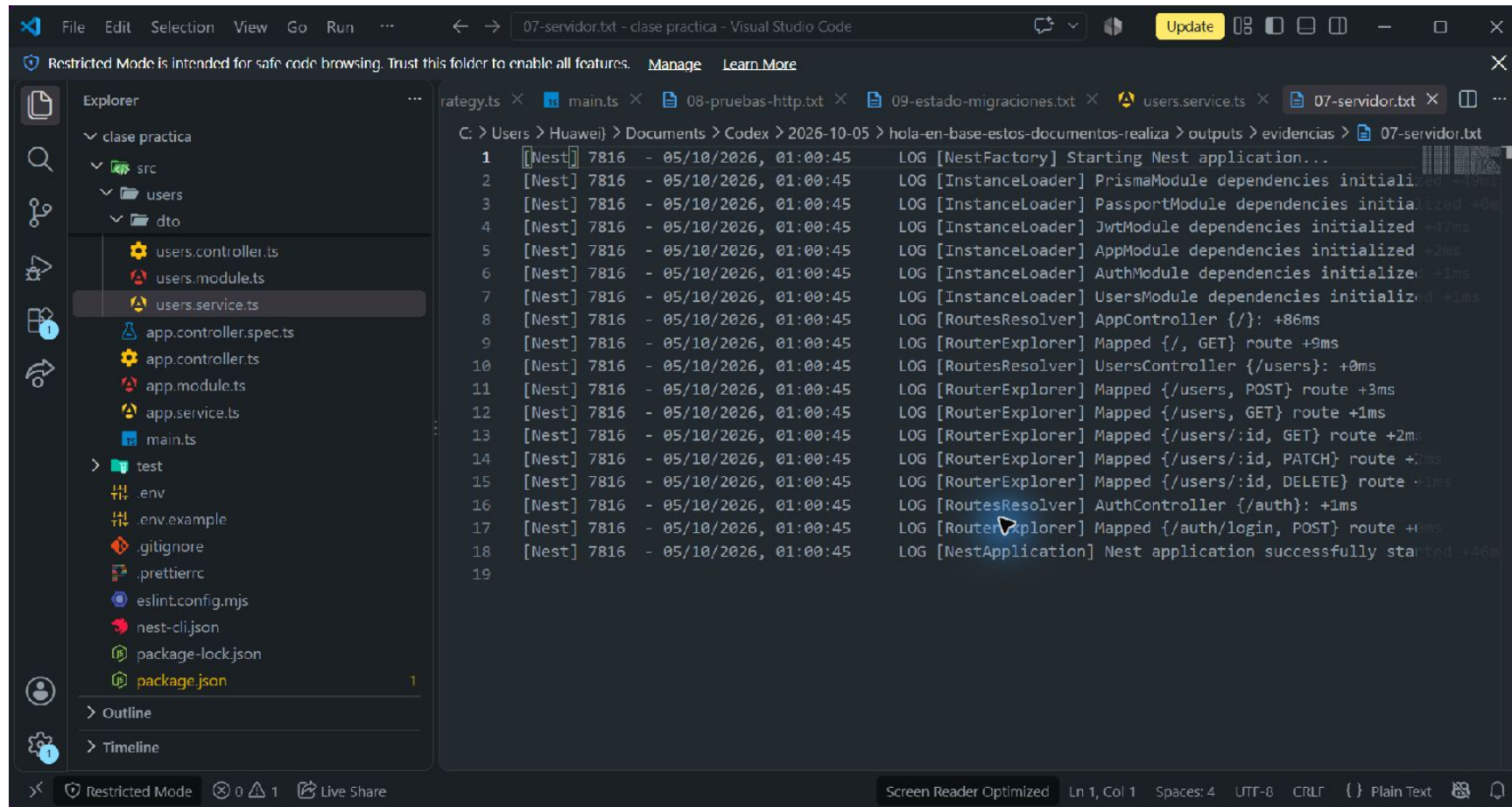
Guía 1, página 4; integración de las tres guías

La compilación terminó correctamente. Se excluyó prisma/ del tsconfig de compilación para generar dist/main.js. El puerto 3000 estaba ocupado por una aplicación ajena: se configuró PORT=3001 y se verificó la práctica de forma independiente. El registro muestra las rutas y Nest application successfully started.

```
npm run build
```

```
npm run start:prod
```

```
Alternativa de desarrollo: npm run start:dev
```



```
07-servidor.txt - clase practica - Visual Studio Code
Restricted Mode is intended for safe code browsing. Trust this folder to enable all features. Manage Learn More

Explorer
  clase practica
    src
      users
        dto
          users.controller.ts
          users.module.ts
          users.service.ts
        app.controller.spec.ts
        app.controller.ts
        app.module.ts
        app.service.ts
        main.ts
      test
        env
        env.example
        .gitignore
        .prettierrc
        eslint.config.mjs
        nest-cli.json
        package-lock.json
        package.json

C: > Users > Huawei > Documents > Codex > 2026-10-05 > hola-en-base-estos-documentos-realiza > outputs > evidencias > 07-servidor.txt
1 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [NestFactory] Starting Nest application...
2 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [InstanceLoader] PrismaModule dependencies initialized +1ms
3 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [InstanceLoader] PassportModule dependencies initialized +0ms
4 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [InstanceLoader] JwtModule dependencies initialized +47ms
5 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [InstanceLoader] AppModule dependencies initialized +2ms
6 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [InstanceLoader] AuthModule dependencies initialized +1ms
7 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [InstanceLoader] UsersModule dependencies initialized +1ms
8 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RoutesResolver] AppController {/}: +86ms
9 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/, GET} route +9ms
10 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RoutesResolver] UsersController {/users}: +0ms
11 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/users, POST} route +3ms
12 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/users, GET} route +1ms
13 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/users/:id, GET} route +2ms
14 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/users/:id, PATCH} route +1ms
15 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +1ms
16 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RoutesResolver] AuthController {/auth}: +1ms
17 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [RouterExplorer] Mapped {/auth/login, POST} route +0ms
18 [Nest] 7816 - 05/10/2026, 01:00:45 LOG [NestApplication] Nest application successfully started +46ms
19
```

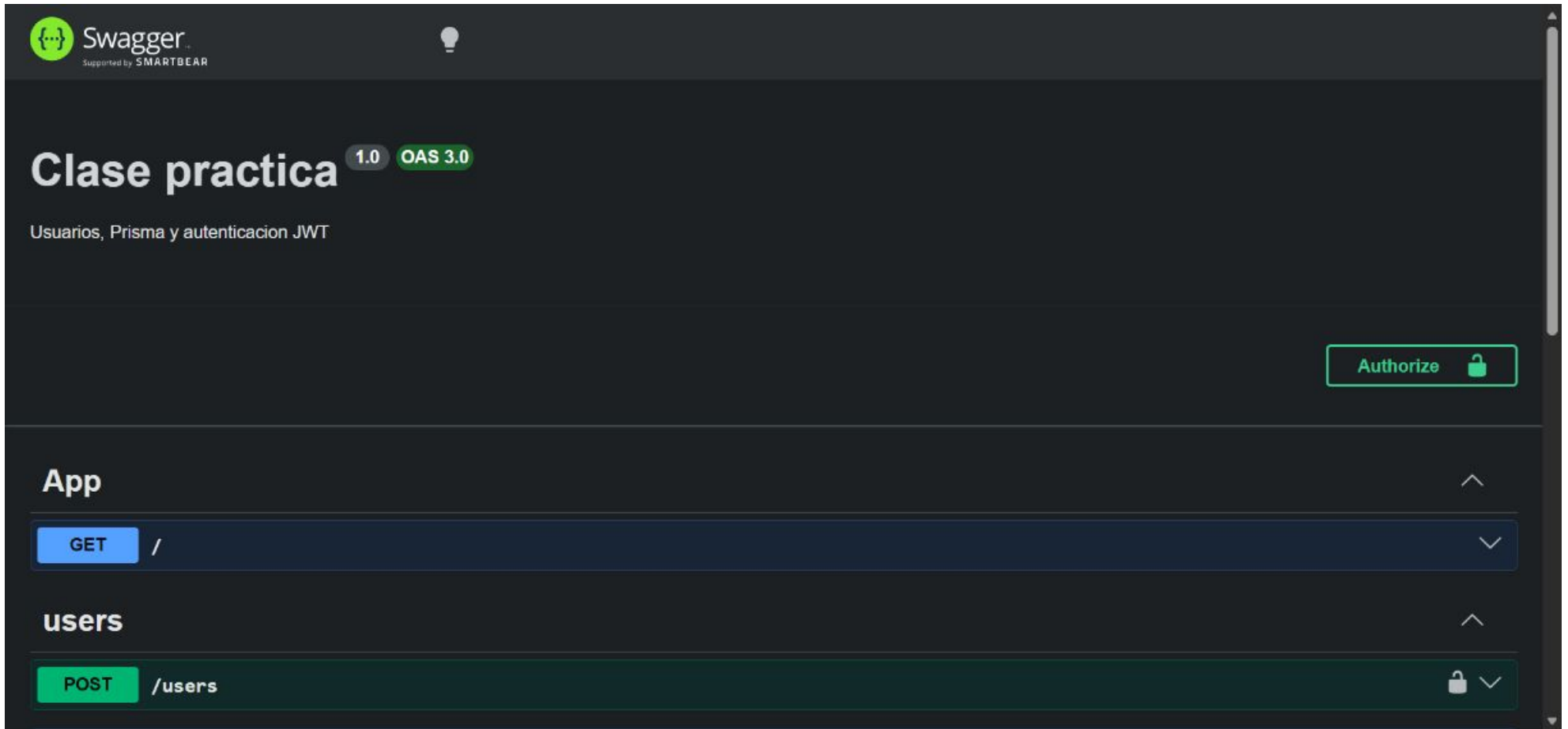
Captura real de ventana o navegador. Archivo: 19-servidor.png

15 Explorar la documentación en Swagger

Guía 2, página 2

Se abrió Swagger en el navegador Edge. Aparecen el endpoint raíz, las cinco rutas de usuarios, el login y los esquemas CreateUserDto, UpdateUserDto y LoginDto. Las pruebas automáticas también confirmaron el documento OpenAPI /api-json.

Abrir <http://127.0.0.1:3001/api>



Captura real de ventana o navegador. Archivo: 10-swagger.png

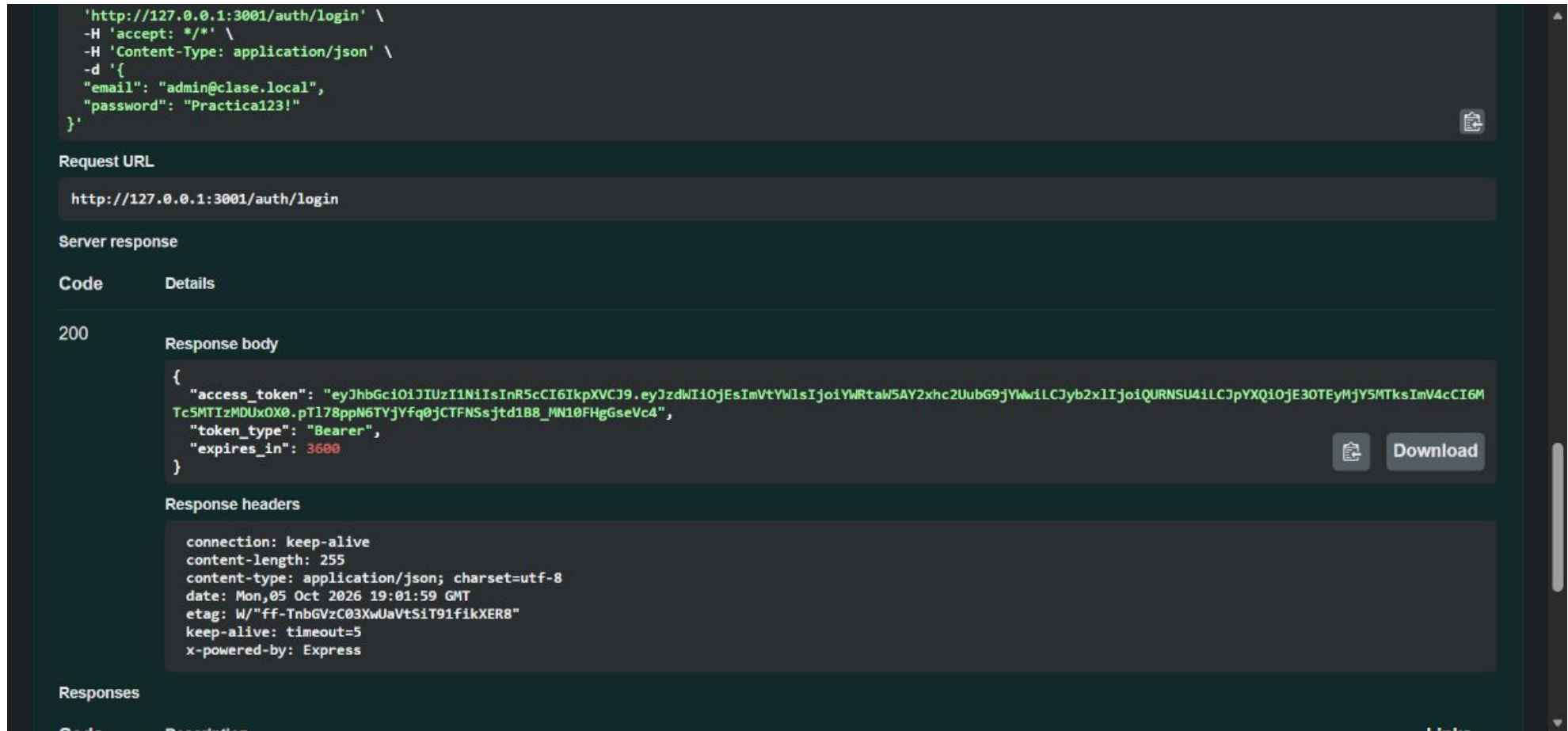
16 Probar el login en Swagger

Guía 3, páginas 7 y 10

En POST /auth/login se pulsó Try it out y Execute con las credenciales locales. La API devolvió HTTP 200 y access_token, token_type Bearer y expires_in 3600. El token de la captura es temporal y corresponde únicamente a la práctica local.

POST /auth/login

```
{"email":"admin@clase.local","password":"Practica123!"}
```



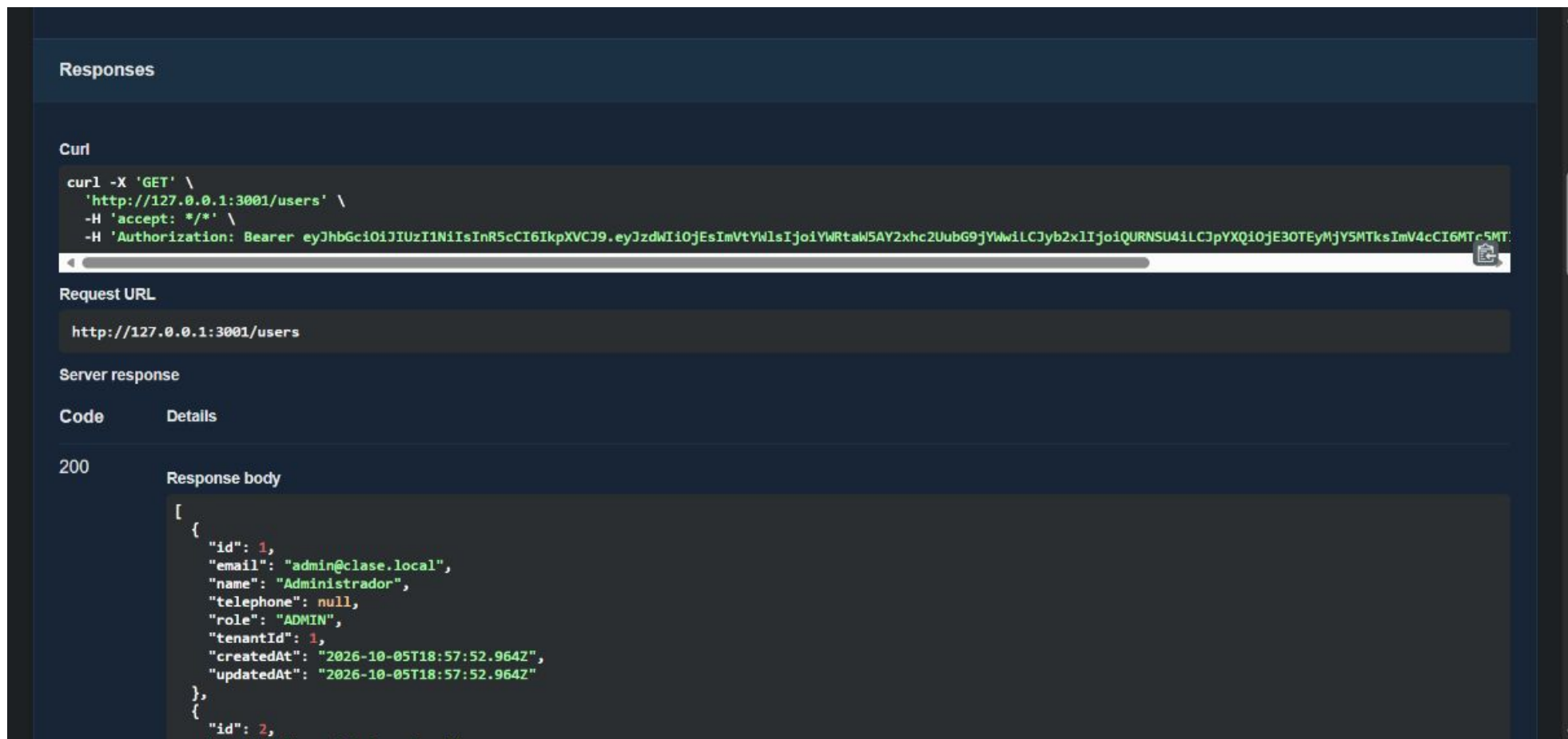
Captura real de ventana o navegador. Archivo: 11-login-jwt.png

17 Consultar usuarios con autorización

Guía 3, páginas 10-11

Se copió el JWT de la respuesta visible, se abrió Authorize y se aplicó el token sin escribir el prefijo Bearer. GET /users devolvió HTTP 200 y los usuarios iniciales. La respuesta muestra el tenantId y no contiene contraseñas ni hashes.

```
Authorize > pegar access_token > Apply credentials
GET /users
```



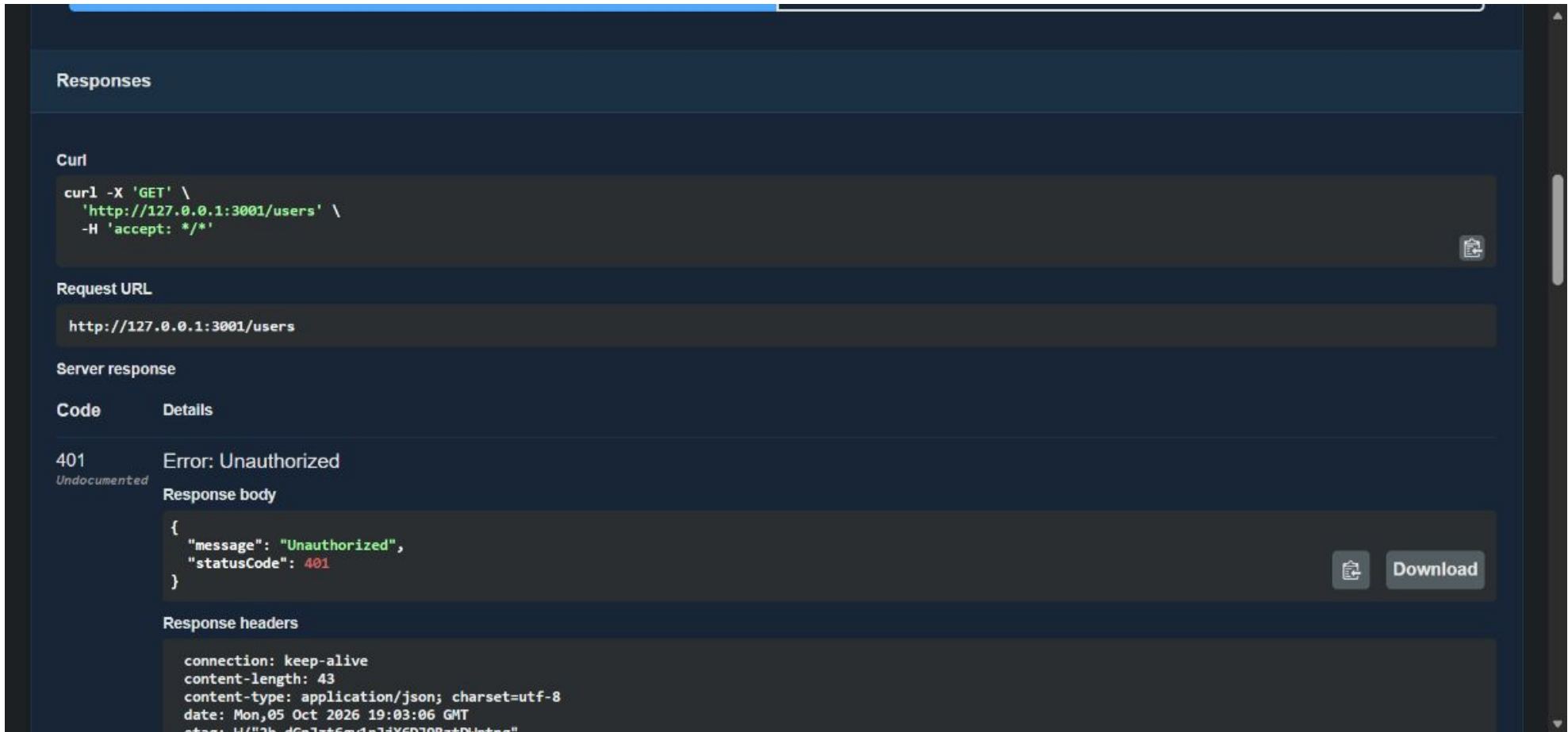
Captura real de ventana o navegador. Archivo: 12-users-autorizado.png

18 Comprobar el rechazo sin token

Guía 3, página 11

Se retiró la autorización y se volvió a ejecutar GET /users. Swagger mostró HTTP 401 con message Unauthorized y statusCode 401, comprobando la protección real de la ruta.

Remove authorization
GET /users



The screenshot displays the Swagger UI interface for a REST API. The 'Responses' tab is selected, showing the details for the 401 status code. The 'Curl' section shows the command used to test the endpoint. The 'Request URL' is 'http://127.0.0.1:3001/users'. The 'Server response' section shows a 401 status code with the message 'Error: Unauthorized'. The 'Response body' is a JSON object: `{ "message": "Unauthorized", "statusCode": 401 }`. The 'Response headers' section shows the following headers: `connection: keep-alive`, `content-length: 43`, `content-type: application/json; charset=utf-8`, `date: Mon, 05 Oct 2026 19:03:06 GMT`, and `etag: W/"2h-d6n1zt6m4n1iY6D19BztDwotng"`.

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:3001/users' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:3001/users
```

Server response

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "message": "Unauthorized", "statusCode": 401 }</pre> Response headers <pre>connection: keep-alive content-length: 43 content-type: application/json; charset=utf-8 date: Mon, 05 Oct 2026 19:03:06 GMT etag: W/"2h-d6n1zt6m4n1iY6D19BztDwotng"</pre>

Captura real de ventana o navegador. Archivo: 13-users-sin-token.png

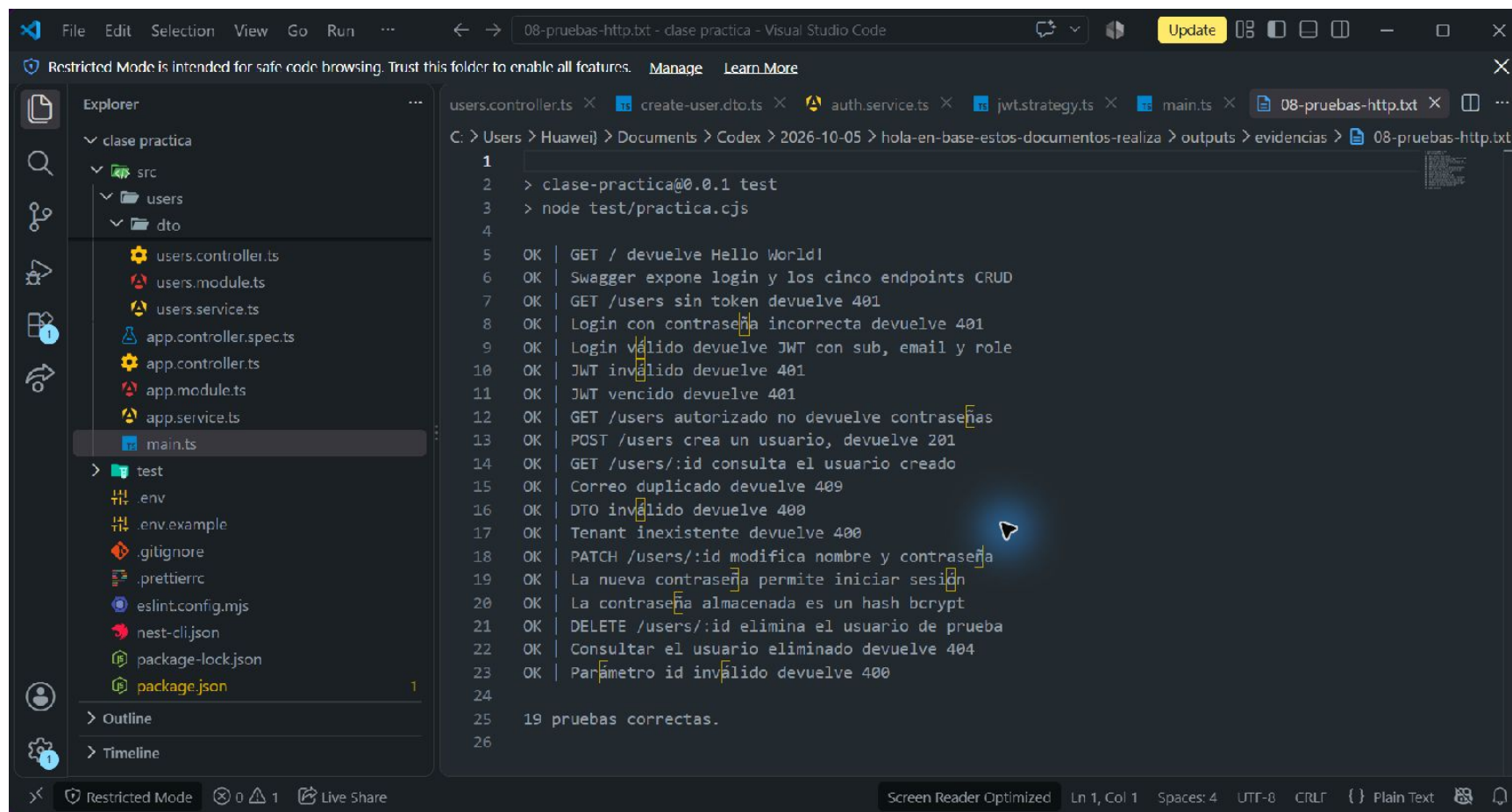
19 Ejecutar pruebas completas de la práctica

Verificación adicional del desarrollo

npm test ejecutó 19 comprobaciones HTTP y de persistencia contra el puerto 3001. Se creó, consultó, modificó y eliminó un usuario temporal. Se comprobó el hash en SQLite y se validaron tokens inválidos y vencidos. Los dos usuarios iniciales permanecen en la base de datos.

Con el servidor encendido, en otra terminal:

npm test



```
C: > Users > Huawei > Documents > Codex > 2026-10-05 > hola-en-base-estos-documentos-realiza > outputs > evidencias > 08-pruebas-http.txt

1
2 > clase-practica@0.0.1 test
3 > node test/practica.cjs
4
5 OK | GET / devuelve Hello World!
6 OK | Swagger expone login y los cinco endpoints CRUD
7 OK | GET /users sin token devuelve 401
8 OK | Login con contraseña incorrecta devuelve 401
9 OK | Login válido devuelve JWT con sub, email y role
10 OK | JWT inválido devuelve 401
11 OK | JWT vencido devuelve 401
12 OK | GET /users autorizado no devuelve contraseñas
13 OK | POST /users crea un usuario, devuelve 201
14 OK | GET /users/:id consulta el usuario creado
15 OK | Correo duplicado devuelve 409
16 OK | DTO inválido devuelve 400
17 OK | Tenant inexistente devuelve 400
18 OK | PATCH /users/:id modifica nombre y contraseña
19 OK | La nueva contraseña permite iniciar sesión
20 OK | La contraseña almacenada es un hash bcrypt
21 OK | DELETE /users/:id elimina el usuario de prueba
22 OK | Consultar el usuario eliminado devuelve 404
23 OK | Parámetro id inválido devuelve 400
24
25 19 pruebas correctas.
26
```

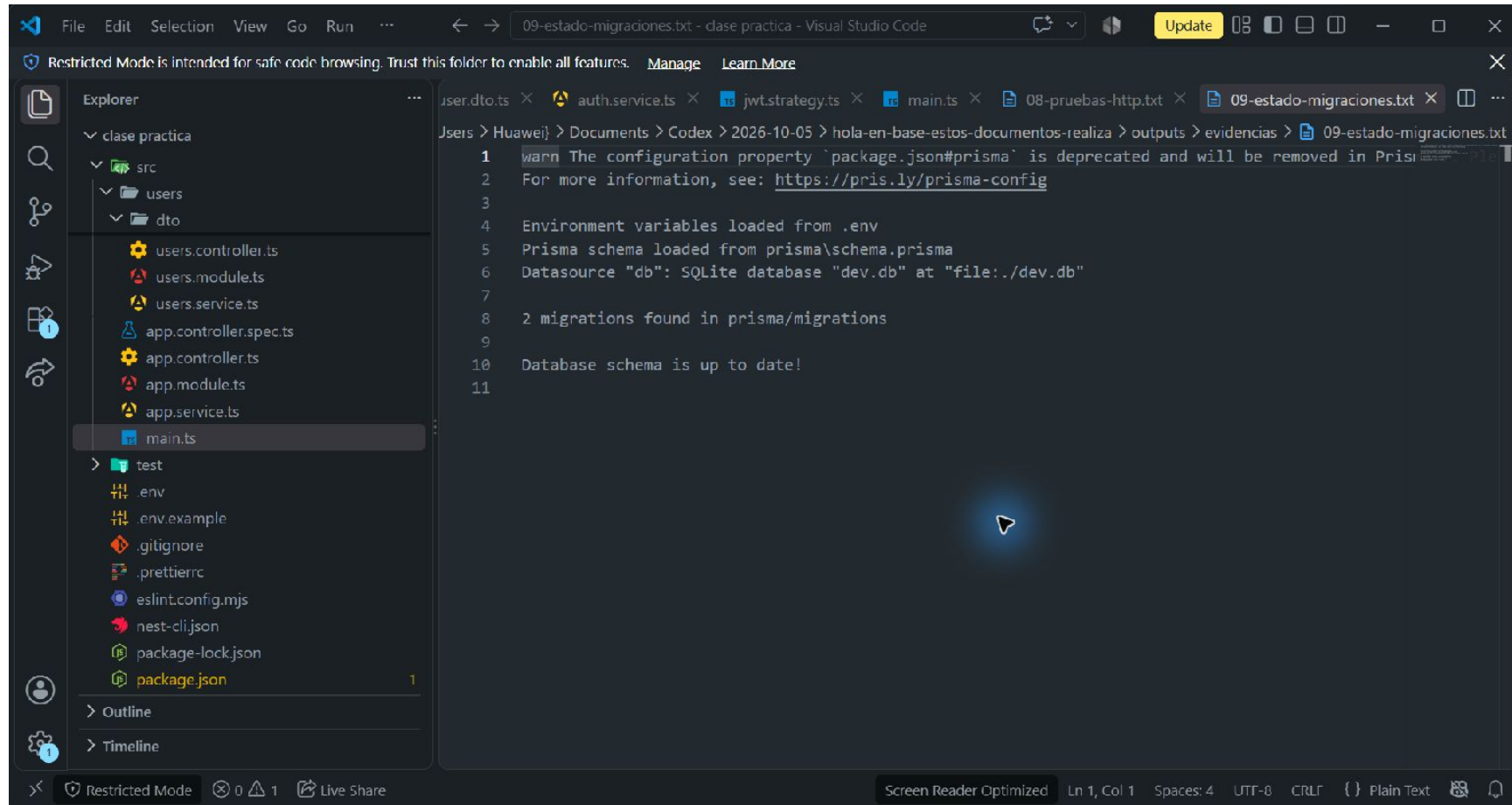
Captura real de ventana o navegador. Archivo: 16-pruebas.png

20 Verificar migraciones y conservar commits

Guía 1, páginas 15 y 17

prisma migrate status confirmó dos migraciones y esquema actualizado. Se crearon tres commits locales: modelo y migraciones, seeder y aplicación con CRUD/JWT/pruebas. No se configuró un remoto ni se envió un enlace a Teams: esos pasos requieren el destino y la cuenta del estudiante.

```
npx prisma migrate status  
git log --oneline
```



Captura real de ventana o navegador. Archivo: 17-migraciones.png

Resultados de las 19 verificaciones

Prueba	Resultado
GET / devuelve Hello World!	OK
Swagger expone login y los cinco endpoints CRUD	OK
GET /users sin token devuelve 401	OK
Login con contraseña incorrecta devuelve 401	OK
Login válido devuelve JWT con sub, email y role	OK
JWT inválido devuelve 401	OK
JWT vencido devuelve 401	OK
GET /users autorizado no devuelve contraseñas	OK
POST /users crea un usuario, devuelve 201	OK
GET /users/:id consulta el usuario creado	OK
Correo duplicado devuelve 409	OK
DTO inválido devuelve 400	OK
Tenant inexistente devuelve 400	OK
PATCH /users/:id modifica nombre y contraseña	OK
La nueva contraseña permite iniciar sesión	OK
La contraseña almacenada es un hash bcrypt	OK
DELETE /users/:id elimina el usuario de prueba	OK
Consultar el usuario eliminado devuelve 404	OK
Parámetro id inválido devuelve 400	OK

Incidencias y decisiones

La descarga inicial del CLI fue lenta; se utilizó un Nest CLI ya instalado y npm instaló las dependencias desde su caché. npm ci detectó un lockfile incompatible durante la preparación; npm install resolvió el árbol y actualizó package-lock.json. SQLite necesitó un archivo vacío antes de la primera migración. Se corrigió la ubicación de dist/main.js mediante tsconfig.build.json. Se cambió de 3000 a 3001 por un puerto ocupado. Una primera comprobación apuntó a la aplicación del puerto 3000 y se descartó; las 19 pruebas finales y las capturas de Swagger corresponden al proyecto del puerto 3001.

Conclusiones y alcance

Se completó el proyecto funcional de las tres guías: esquema relacional, migraciones, datos iniciales, servicio Prisma compartido, CRUD con validación, Swagger y JWT. Las 19 verificaciones finales fueron correctas. La práctica comprueba autenticación; el rol del JWT no impone permisos por rol y la relación tenant no implementa aislamiento de datos. Las credenciales son de demostración local. Los commits son locales; la entrega a Teams queda a cargo del estudiante con su cuenta y el destino correspondiente.

Repetir la práctica

El ZIP incluye el código, los commits y las evidencias; excluye node_modules, dist, la base de datos y .env. Extraer, abrir clase practica y seguir README.md: npm ci; copiar .env.example a .env; generar un JWT_SECRET; prisma migrate deploy; prisma generate; prisma db seed; npm run build; npm run start:prod. En otra terminal, npm test. Para modificar el esquema se utiliza migrate dev; para aplicar las migraciones existentes en una copia nueva se utiliza migrate deploy.

Credenciales iniciales: admin@clase.local o usuario@clase.local; contraseña Practica123!.

Historial local

86f9b41 Guías 2 y 3: CRUD, Swagger, JWT y pruebas HTTP

cbd5705 Guía 1: datos iniciales con bcrypt

5587d95 Guía 1: modelo User y Tenant con migraciones

Fuentes

GD_S7_S1.pdf: guía 1, 17 páginas. GD_S7_S2.pdf: guía 2, 6 páginas. GD_S8_S1_FinalNode.pdf: guía 3, 11 páginas. Consulta técnica: <https://docs.nestjs.com/recipes/prisma> y <https://docs.nestjs.com/security/authentication>. Se conservó Prisma 6 para mantener el estilo de configuración de las guías; la receta oficial consultada actualmente explica Prisma 7. Los comandos propuestos por las guías se corrigieron cuando requerían sintaxis válida, por ejemplo `npx prisma migrate dev --name init`. Las instrucciones de publicar y enviar a Teams contenidas en los PDF no se trataron como una autorización para comunicar datos a terceros.